

Listas não-lineares

ÁRVORE BINÁRIA DE BUSCA

Resolução de Problemas
Estruturados em Computação



Profa. Lisiane Reips

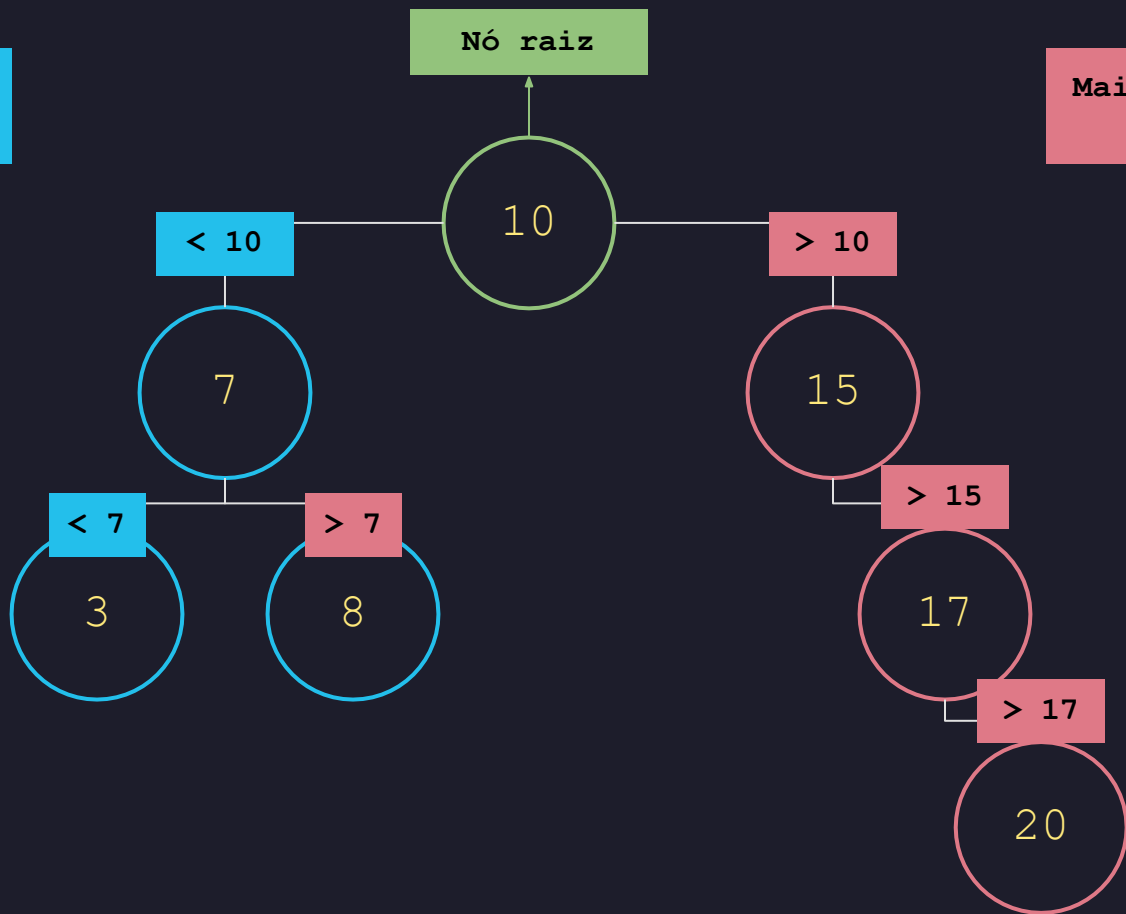
1

INSERÇÃO

de Elemento

Menores para a esquerda

Maiores para a direita

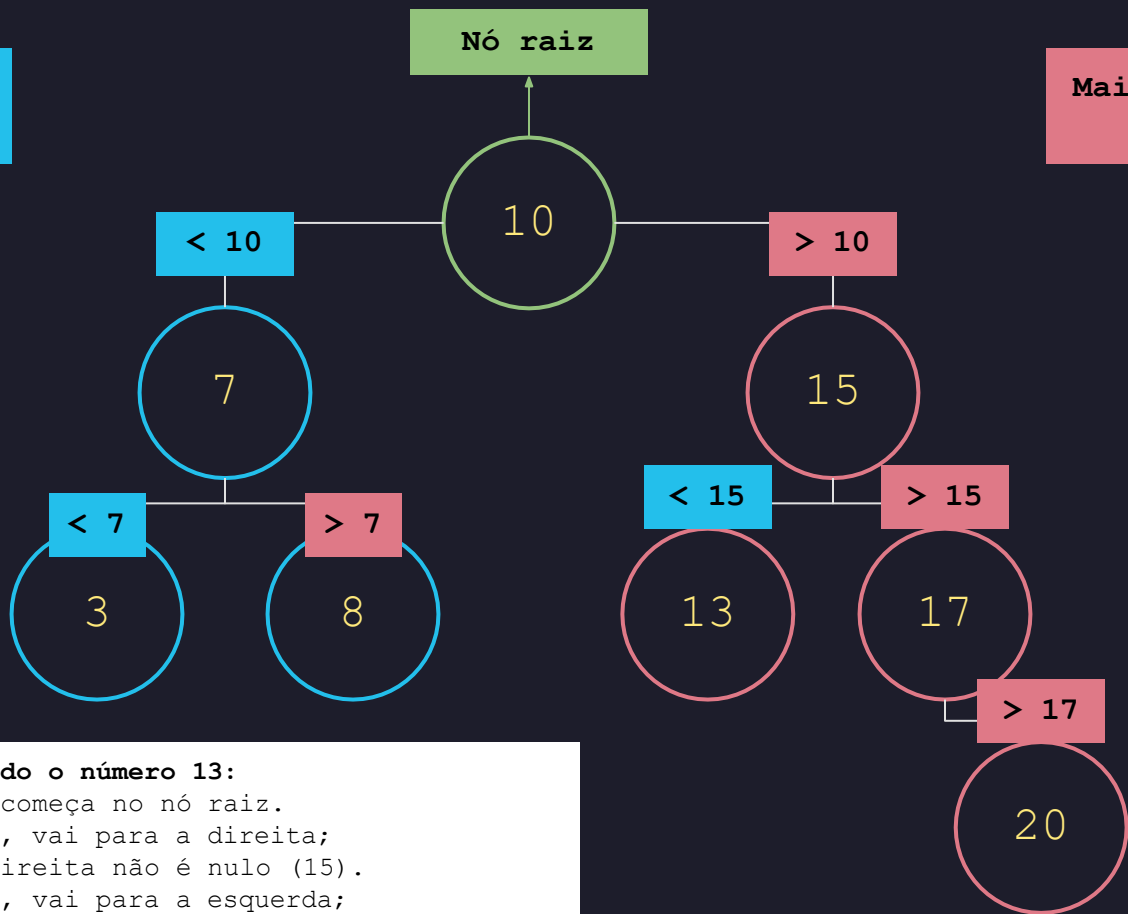


Ordem de inserção:

1. 10 (raiz)
2. 15 (> 10)
3. 17 (> 15)
4. 7 (< 10)
5. 3 (< 7)
6. 20 (> 17)
7. 8 (> 7)

Menores para a esquerda

Maiores para a direita



Inserindo o número 13:

Sempre começa no nó raiz.

13 > 10, vai para a direita;

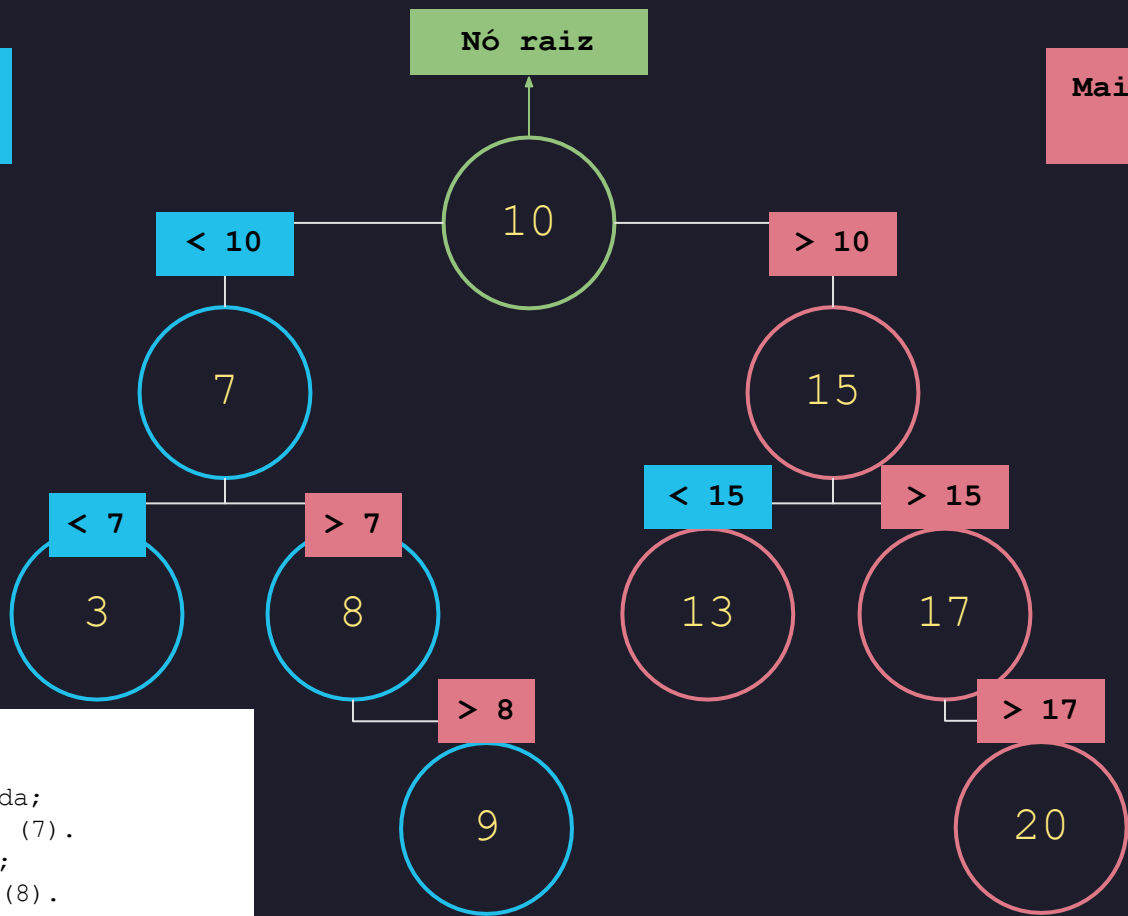
Nó da direita não é nulo (15).

13 < 15, vai para a esquerda;

Nó da esquerda é nulo, insere na esquerda.

Menores para a esquerda

Maiores para a direita



Inserindo o número 9:

Sempre começa no nó raiz.

9 < 10, vai para a esquerda;

Nó da esquerda não é nulo (7).

9 > 7, vai para a direita;

Nó da direita não é nulo (8).

9 > 8, vai para a direita;

Nó da direita é nulo, insere na direita.

2

REMOÇÃO

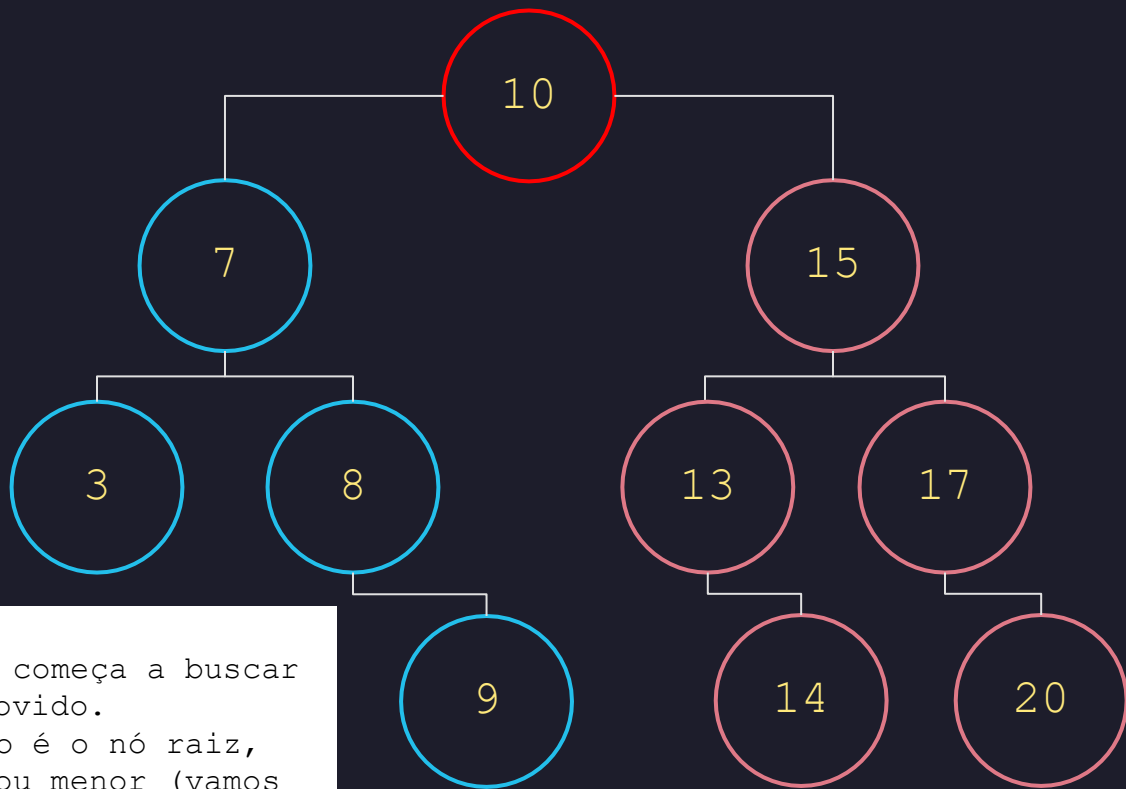
de Elemento



REMOÇÃO

de Nó Folha

Caso 1: Remoção de um elemento folha



Removendo o 9:

A partir do nó raiz, começa a buscar o elemento a ser removido.

Como o elemento 9 não é o nó raiz, verifica se é maior ou menor (vamos para a esquerda ou direita?)

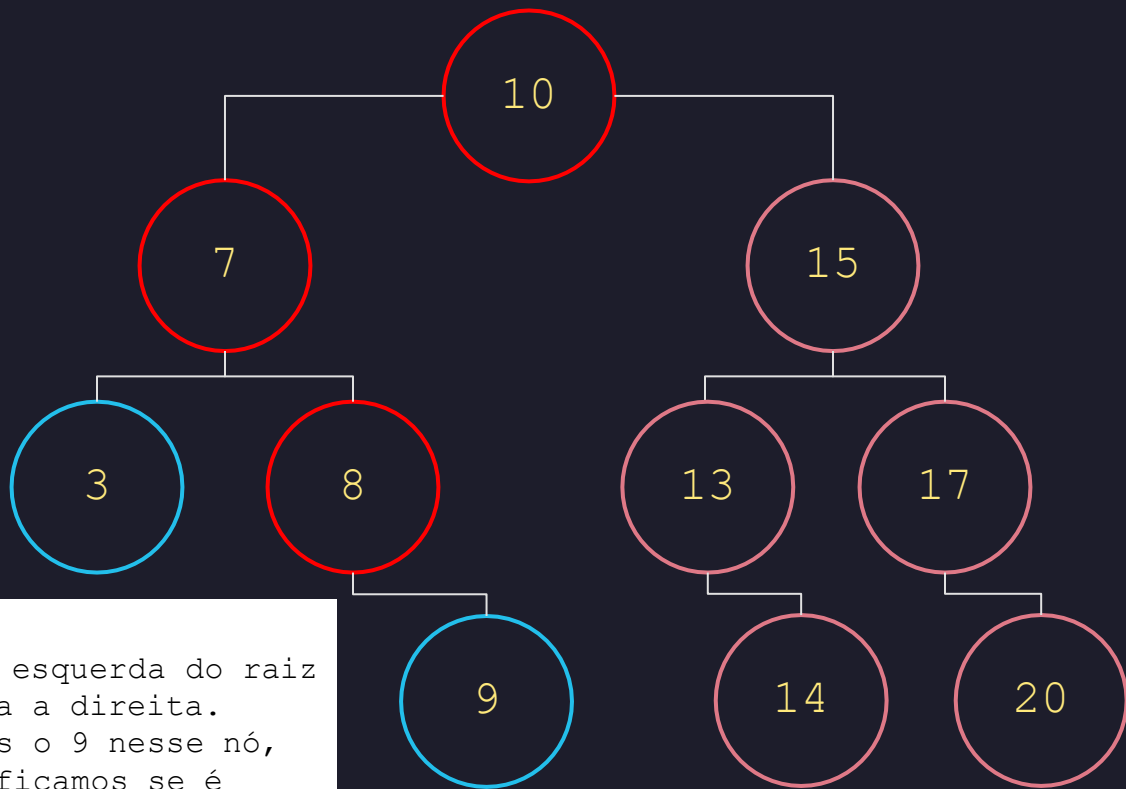
Caso 1: Remoção de um elemento folha



Removendo o 9:

9 é menor que o nó raiz, então vamos verificar o nó à esquerda.
O elemento 9 não é o nó à esquerda do raiz, então verificamos se é maior ou menor (esquerda ou direita?)

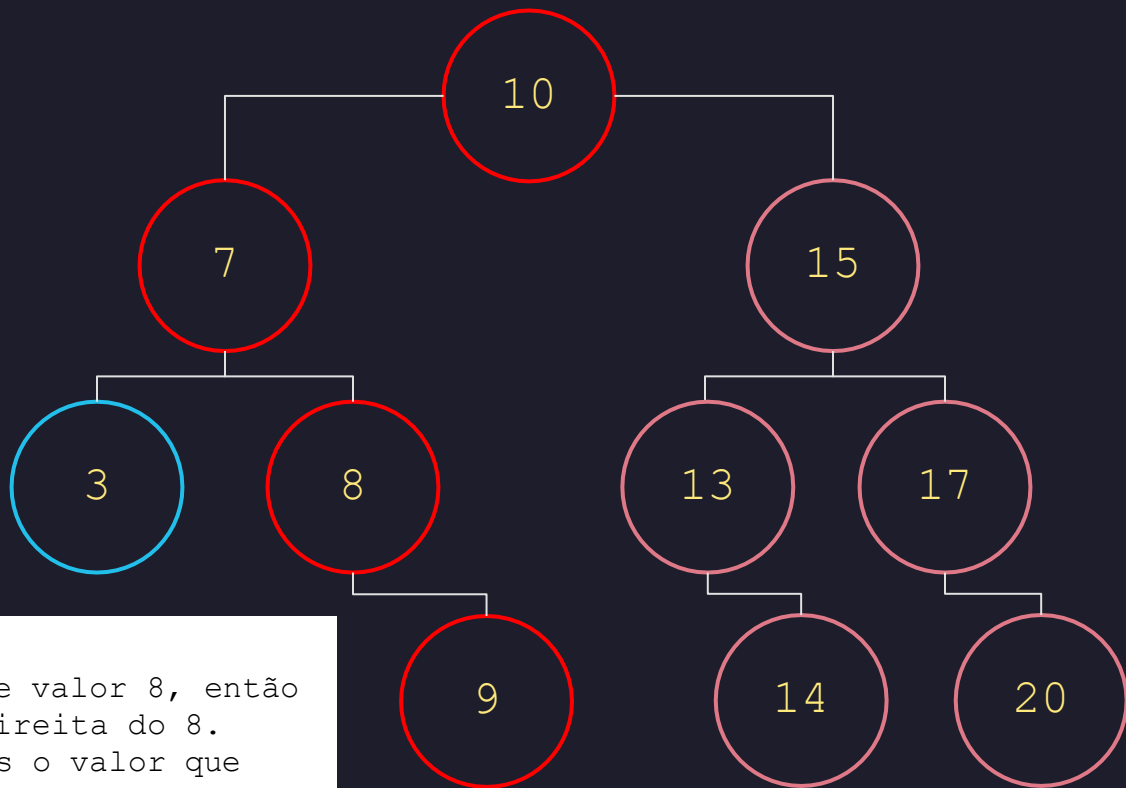
Caso 1: Remoção de um elemento folha



Removendo o 9:

9 é maior que o nó à esquerda do raiz (7), então vamos para a direita. Ainda não encontramos o 9 nesse nó, então novamente verificamos se é maior ou menor (esquerda ou direita)

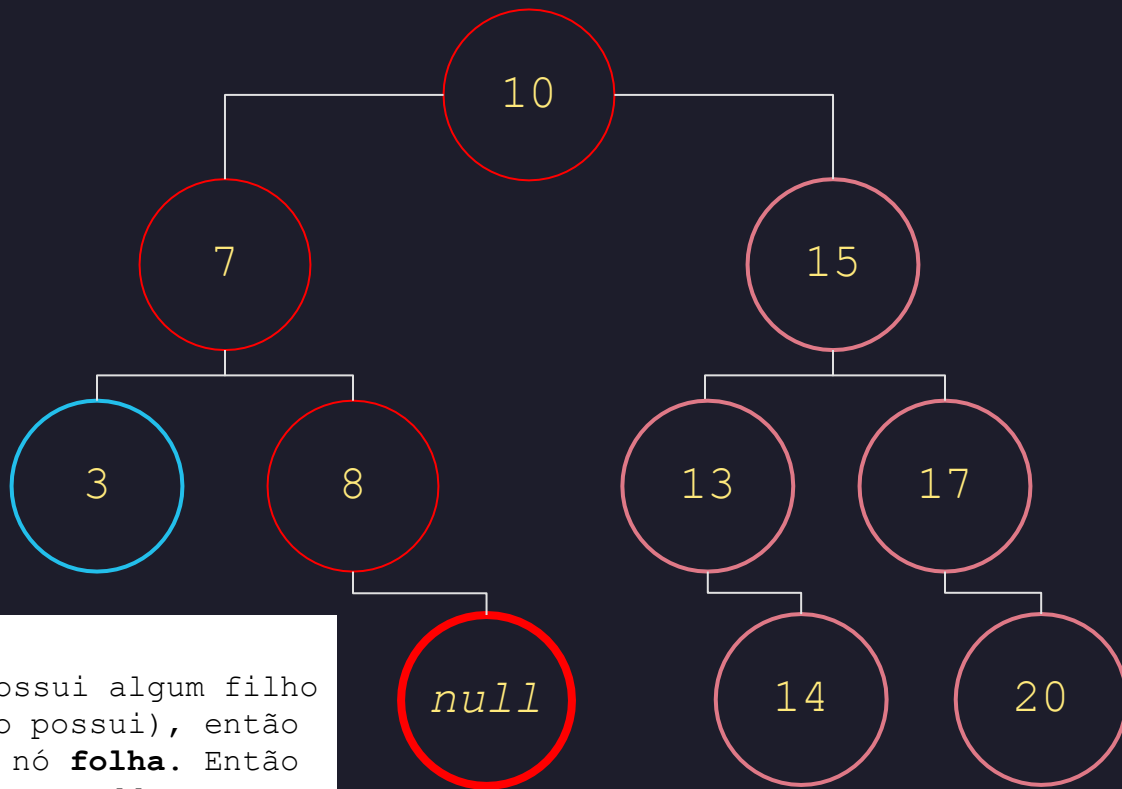
Caso 1: Remoção de um elemento folha



Removendo o 9:

9 é maior que o nó de valor 8, então verificamos o nó à direita do 8. Com isso, encontramos o valor que queremos remover!

Caso 1: Remoção de um elemento folha



Removendo o 9:

Verificamos se ele possui algum filho (o que nesse caso não possui), então sabemos que o 9 é um nó **folha**. Então basta alterar o nó para *null*.



REMOÇÃO

de Nó com 1 Filho

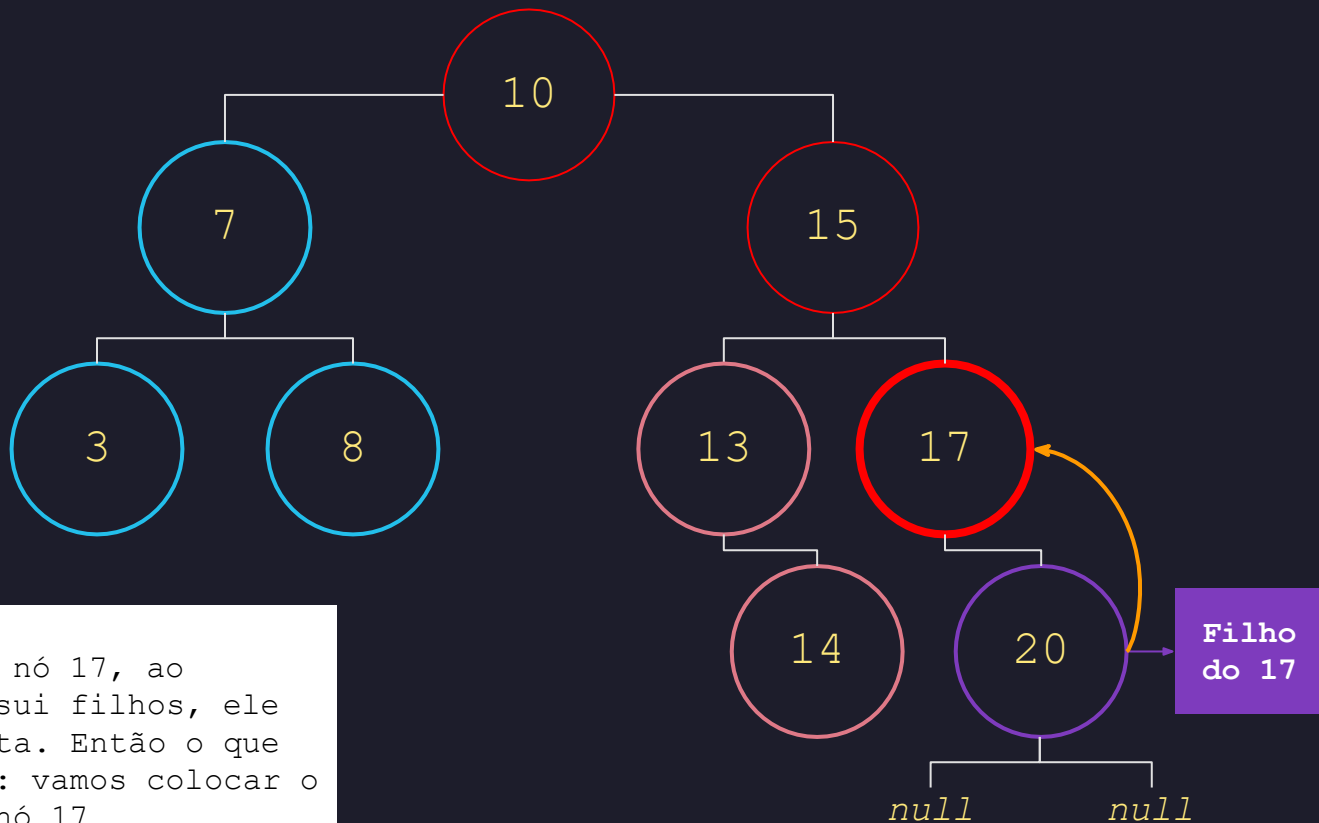
Caso 2: Remoção de um elemento com 1 único filho



Removendo o 17:

O mesmo processo de busca realizado no "Caso 1" acontece para encontrar o nó que desejamos remover.

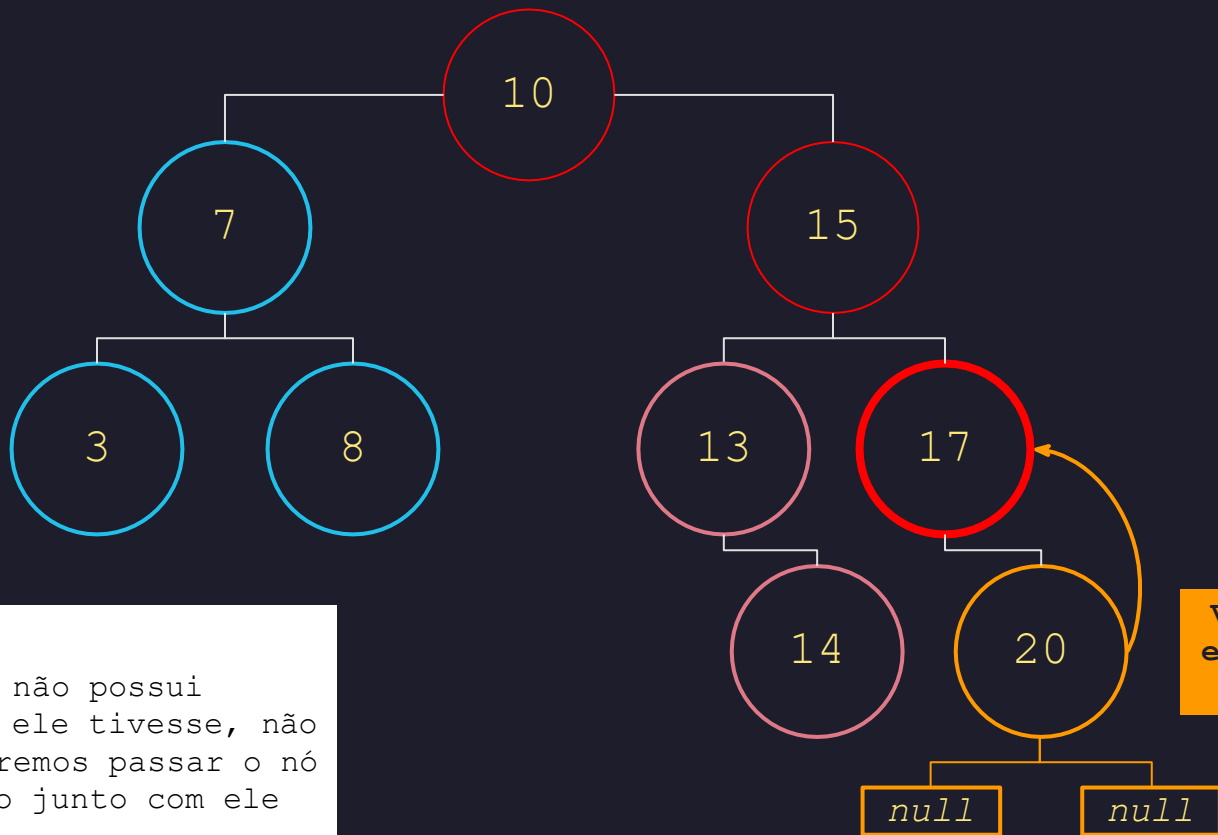
Caso 2: Remoção de um elemento com 1 único filho



Removendo o 17:

A diferença é que no nó 17, ao verificar se ele possui filhos, ele tem um filho à direita. Então o que vai acontecer aqui é: vamos colocar o nó 20 na posição do nó 17

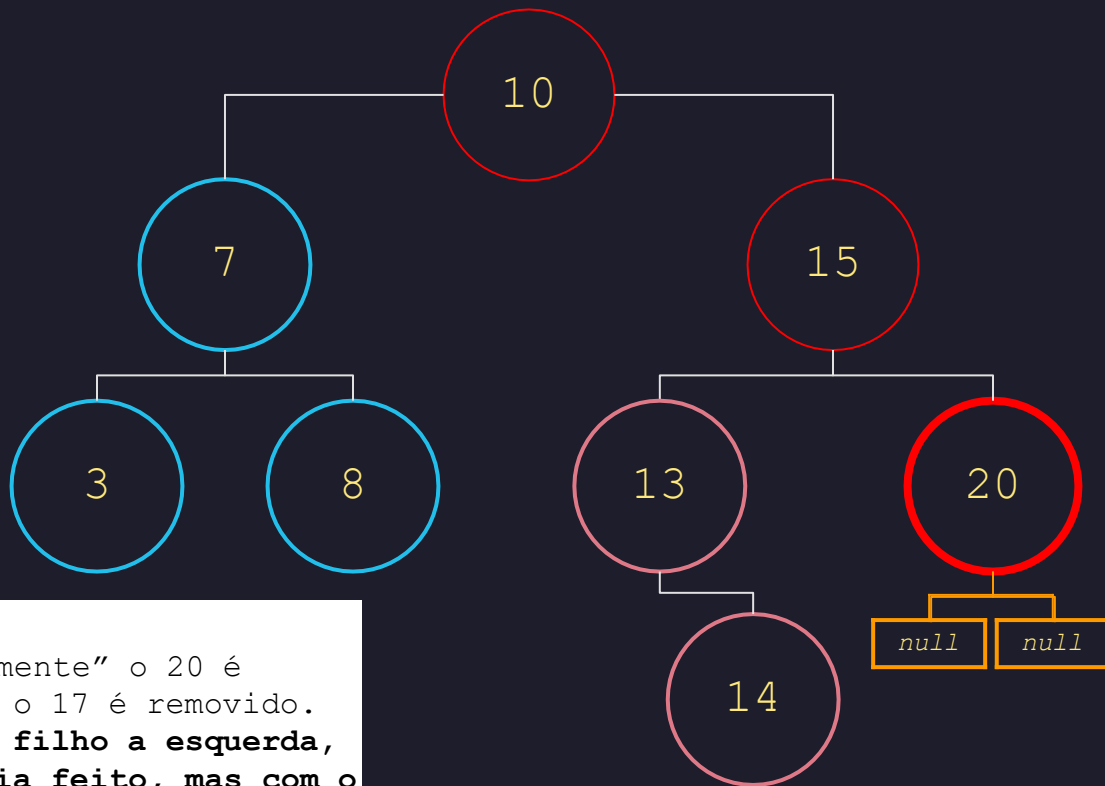
Caso 2: Remoção de um elemento com 1 único filho



Removendo o 17:

Percebiam que o nó 20 não possui filhos, mas mesmo se ele tivesse, não tem problema, pois iremos passar o nó e todos os nós abaixo junto com ele

Caso 2: Remoção de um elemento com 1 único filho



Removendo o 17:

Dessa forma, "magicamente" o 20 é mantido na árvore, e o 17 é removido.

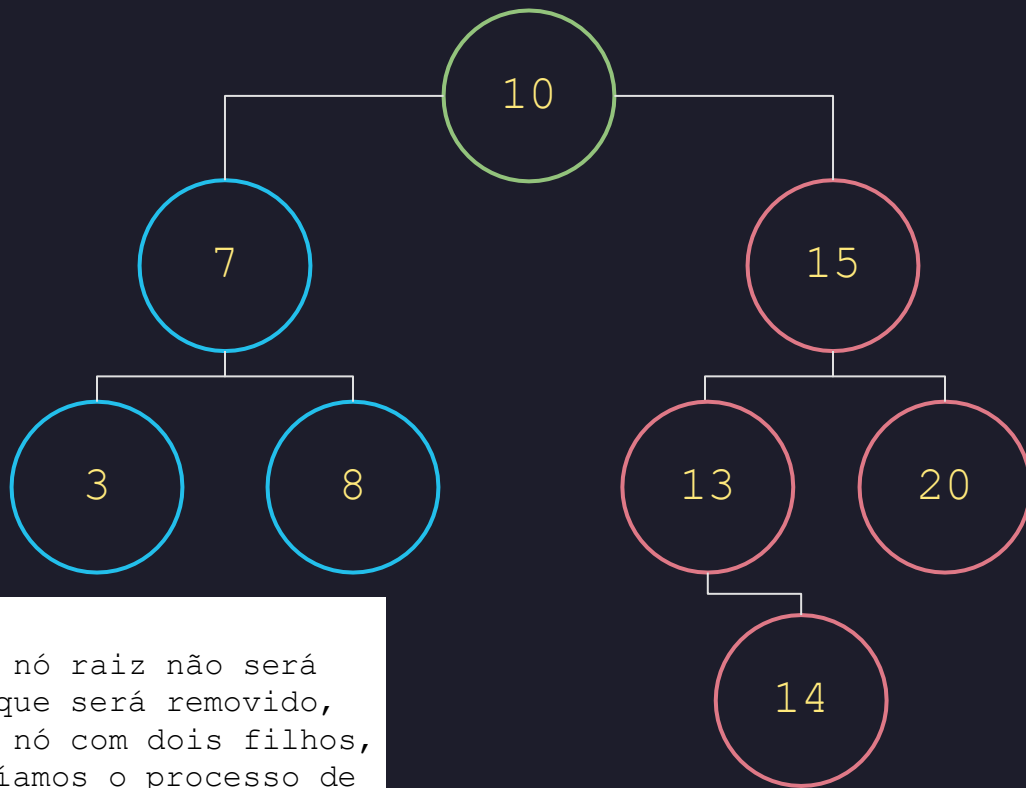
Caso o 17 tivesse um filho a esquerda, o mesmo processo seria feito, mas com o filho da esquerda



REMOÇÃO

de Nó com 2 Filhos

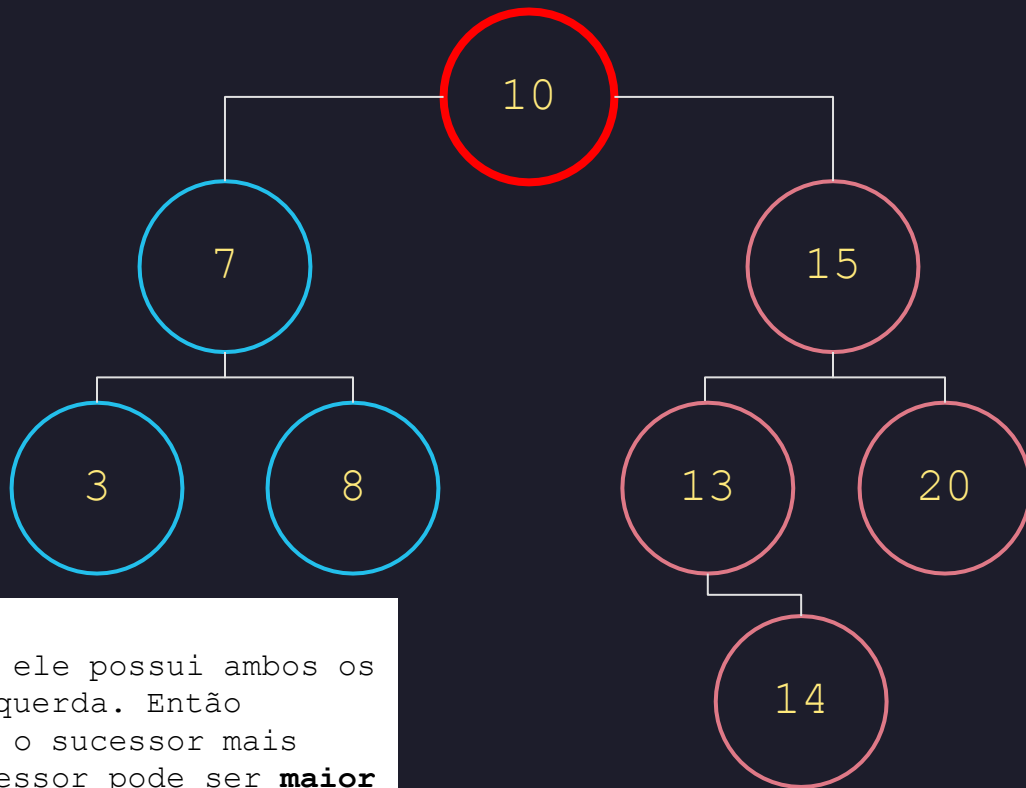
Caso 3: Remoção de um elemento com 2 filhos



Removendo o 10:

Como vamos remover o nó raiz não será preciso buscar o nó que será removido, mas caso fosse outro nó com dois filhos, por exemplo o 7, faríamos o processo de busca igual nos outros casos.

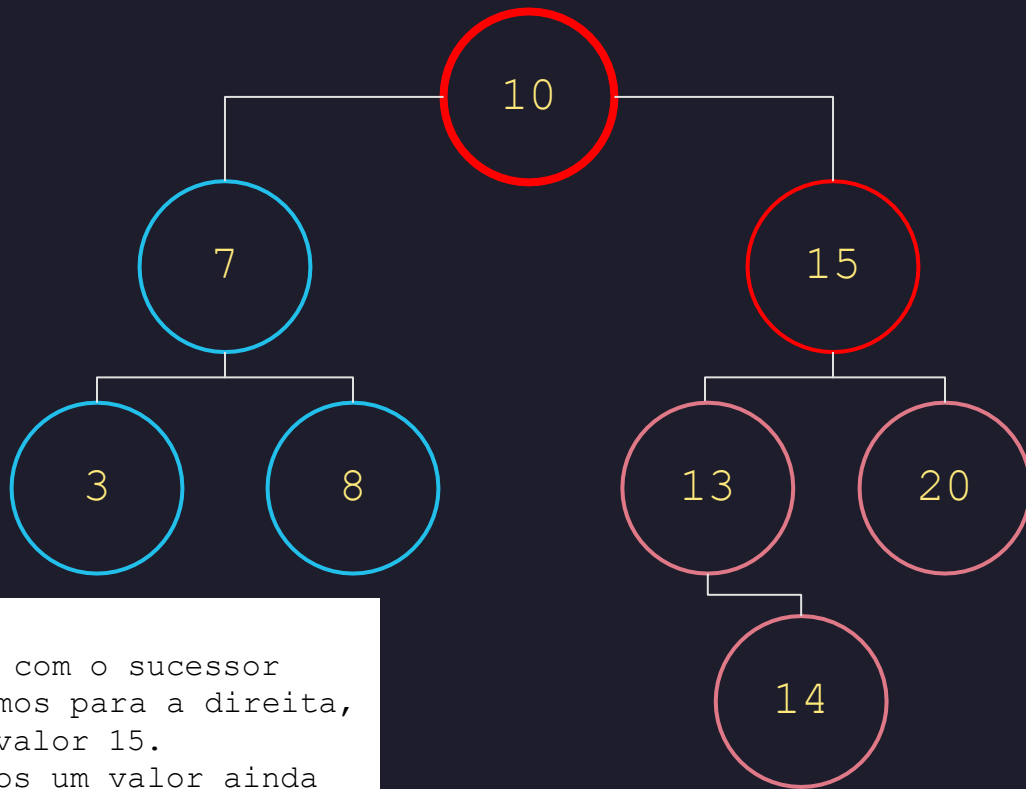
Caso 3: Remoção de um elemento com 2 filhos



Removendo o 10:

Aqui verificamos que ele possui ambos os filhos, direita e esquerda. Então precisamos encontrar o sucessor mais próximo a ele. O sucessor pode ser **maior** ou **menor**, depende da sua implementação.

Caso 3: Remoção de um elemento com 2 filhos

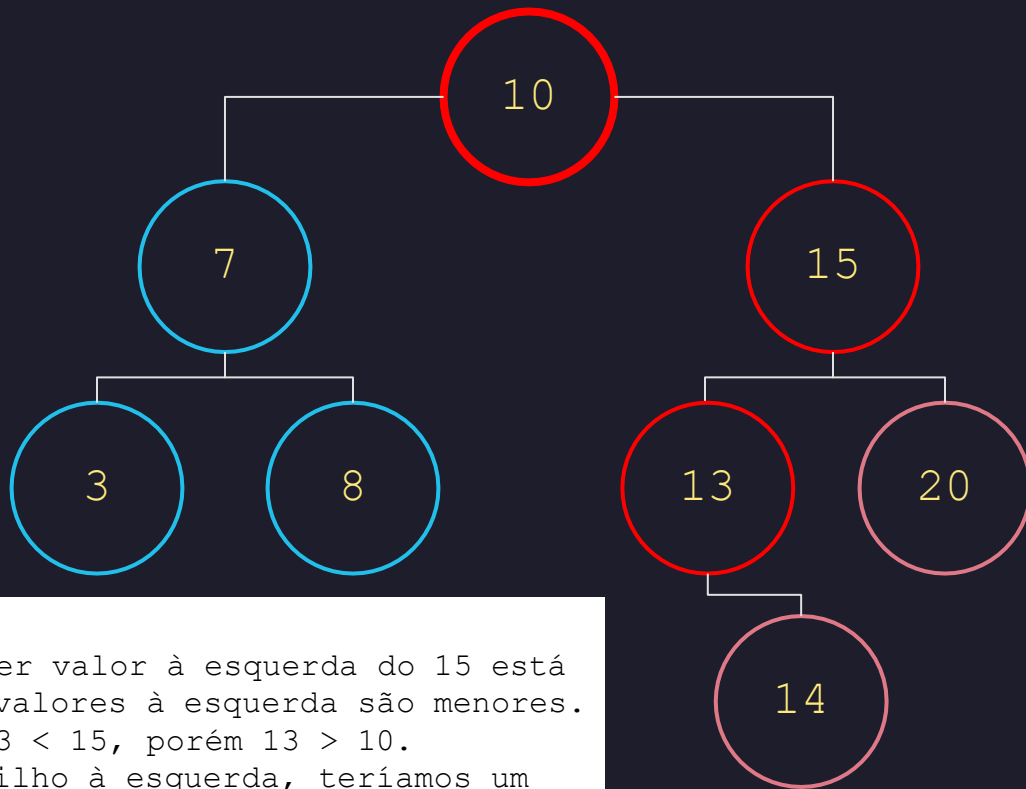


Removendo o 10:

Vamos fazer primeiro com o sucessor **maior**. Se é maior vamos para a direita, então encontramos o valor 15. Mas será que não temos um valor ainda mais próximo? Vamos verificar!

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

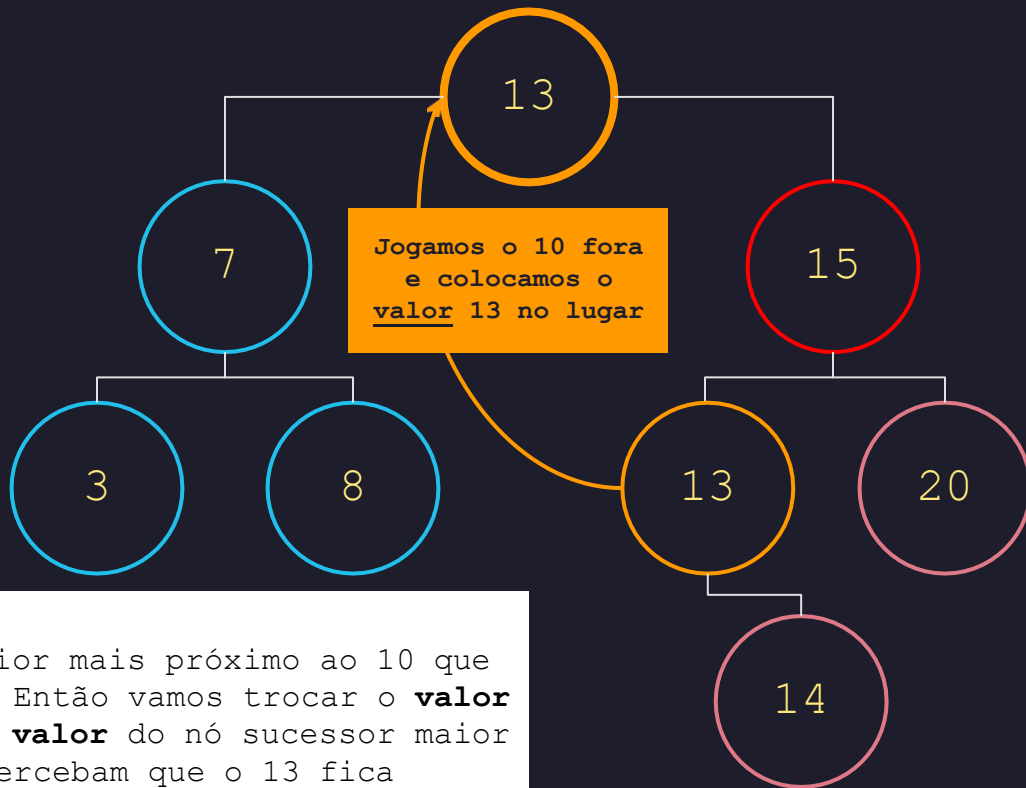


Removendo o 10:

Concordam que qualquer valor à esquerda do 15 está entre 10 e 15? Pois valores à esquerda são menores. Perceba: $15 > 10$ e $13 < 15$, porém $13 > 10$. Se o 13 tivesse um filho à esquerda, teríamos um valor ainda mais próximo. Mas não é o caso.

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

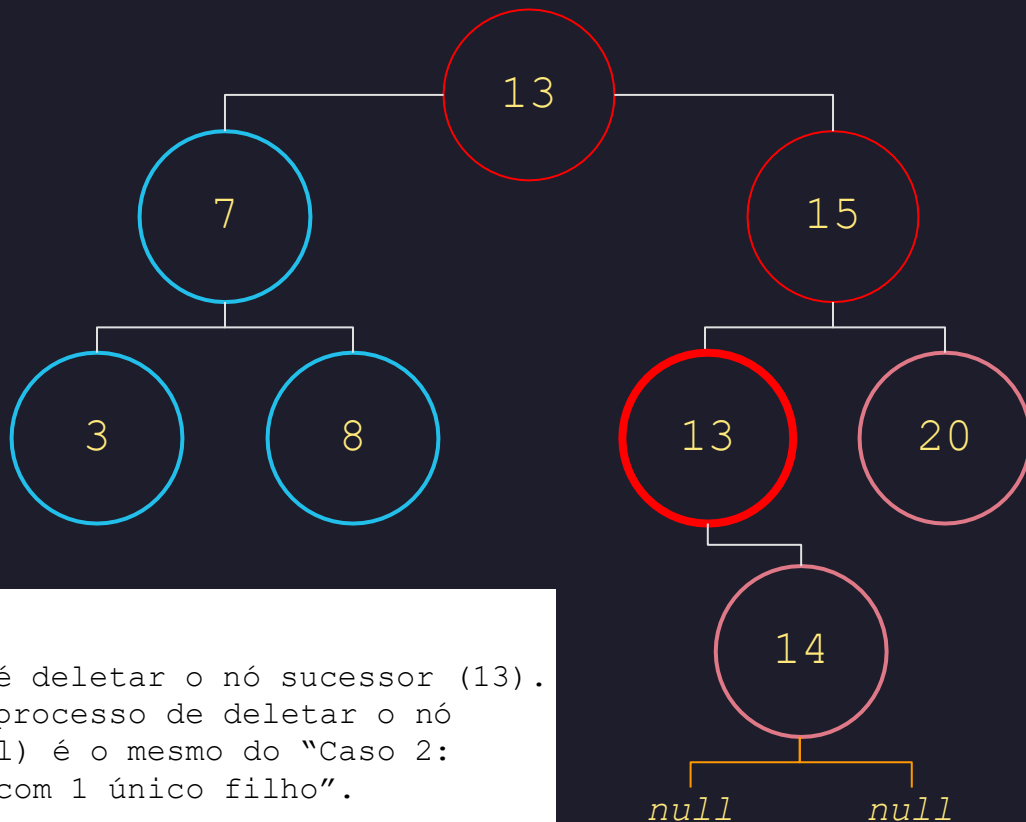


Removendo o 10:

O nó 13 é o valor maior mais próximo ao 10 que temos nesse exemplo. Então vamos trocar o **valor** do nó raiz (10) pelo **valor** do nó sucessor maior mais próximo (13). Percebam que o 13 fica duplicado na árvore, mas é isso mesmo.

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

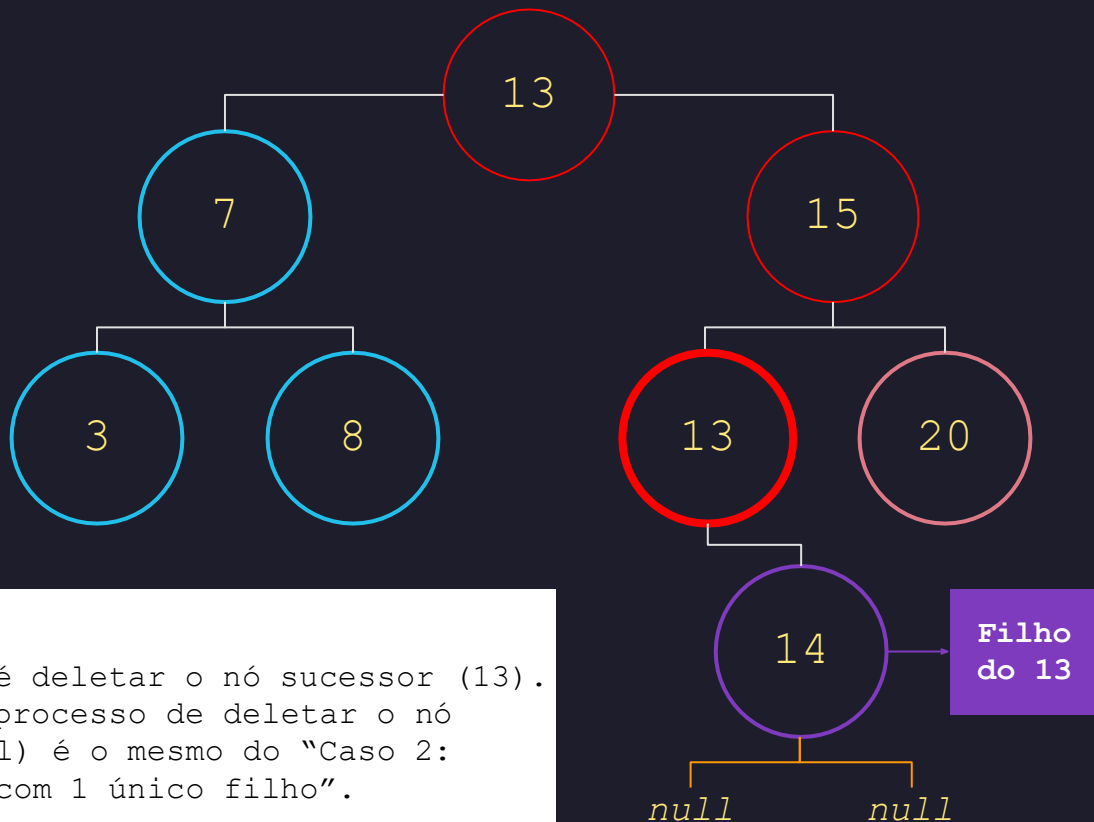


Removendo o 10:

O que fazemos agora é deletar o nó sucessor (13). E vejam que agora o processo de deletar o nó sucessor (13 original) é o mesmo do "Caso 2: remoção de elemento com 1 único filho".

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

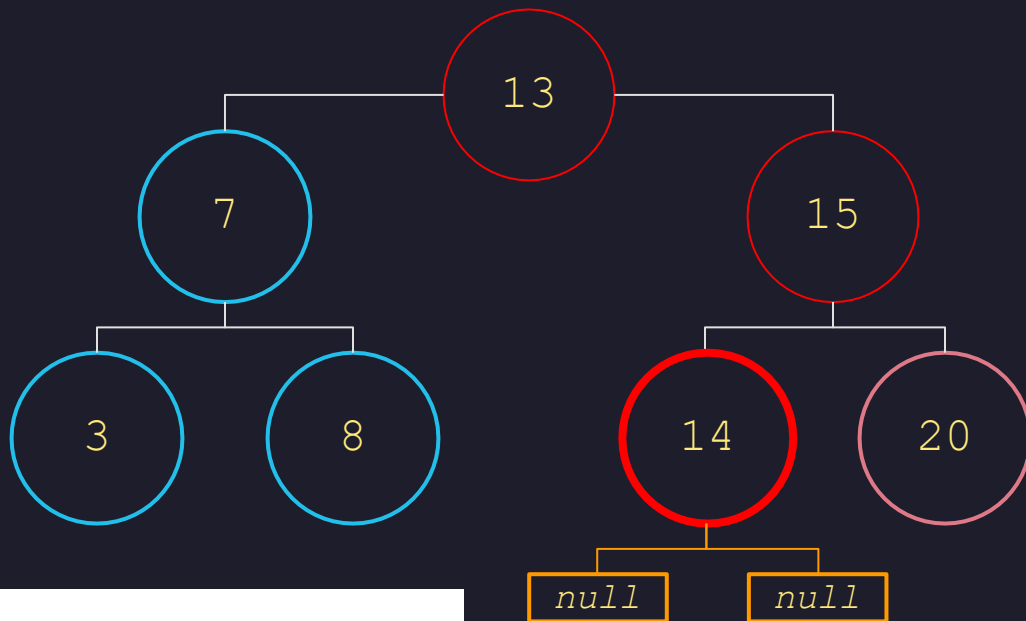


Removendo o 10:

O que fazemos agora é deletar o nó sucessor (13). E vejam que agora o processo de deletar o nó sucessor (13 original) é o mesmo do "Caso 2: remoção de elemento com 1 único filho".

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

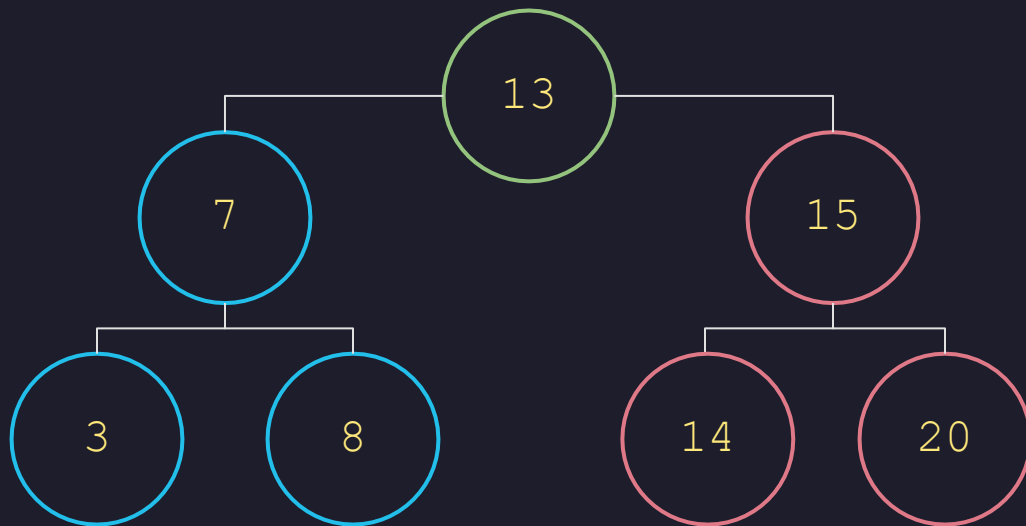


Removendo o 10:

Colocamos o **nó** 14 no lugar do 13 e desta forma removemos o sucessor do nó 10 (13), mantendo a regra da árvore binária de busca de manter os menores à esquerda e maiores à direita.

MAIOR

Caso 3: Remoção de um elemento com 2 filhos



Removendo o 10:

Esse é o resultado da nossa árvore sem o valor 10, considerando o sucessor como o **maior valor** mais próximo.

MAIOR

Caso 3: Remoção de um elemento com 2 filhos

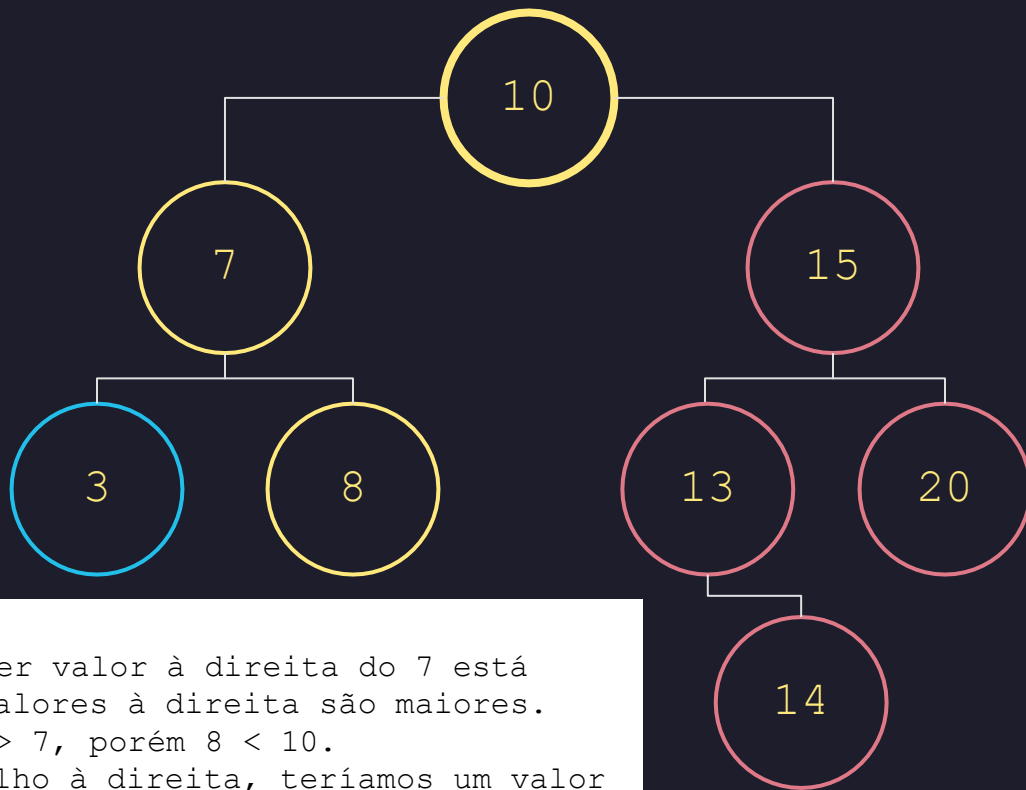


Removendo o 10:

Agora vamos fazer a mesma coisa mas com o sucessor **menor**. Se é menor vamos para a esquerda, então encontramos o valor 7. Mas será que não temos um valor ainda mais próximo? Vamos verificar!

MENOR

Caso 3: Remoção de um elemento com 2 filhos



Removendo o 10:

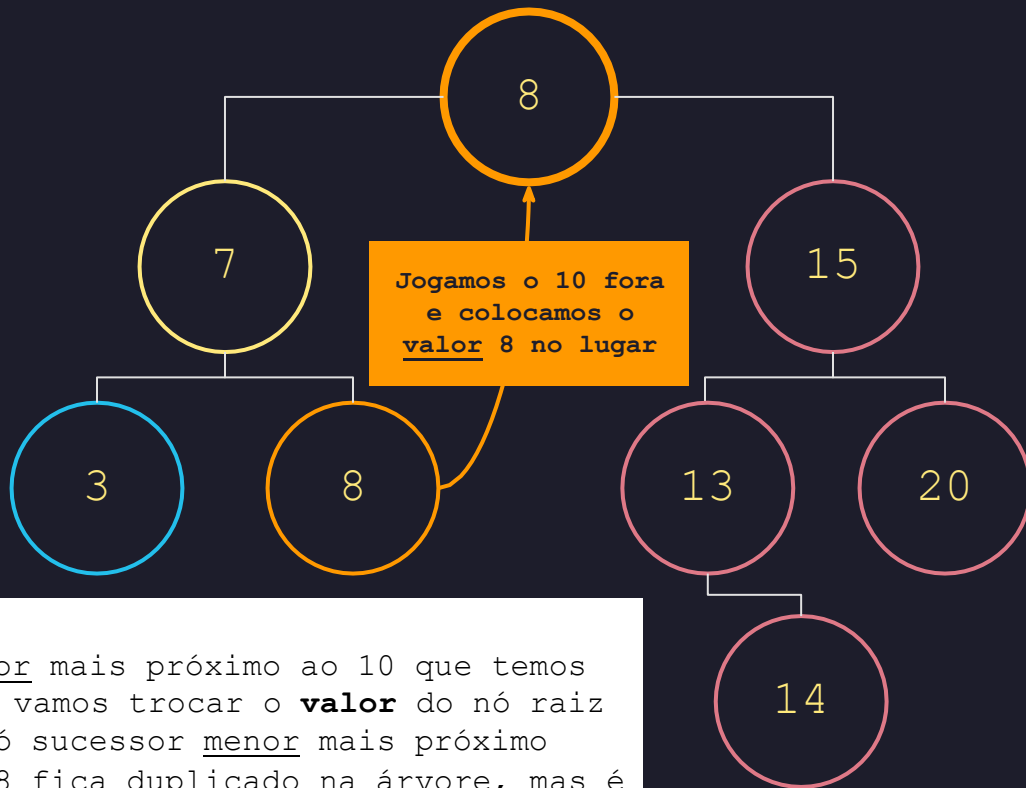
Concordam que qualquer valor à direita do 7 está entre 10 e 7? Pois valores à direita são maiores.

Perceba: $7 < 10$ e $8 > 7$, porém $8 < 10$.

Se o 8 tivesse um filho à direita, teríamos um valor ainda mais próximo. Mas não é o caso.

MENOR

Caso 3: Remoção de um elemento com 2 filhos

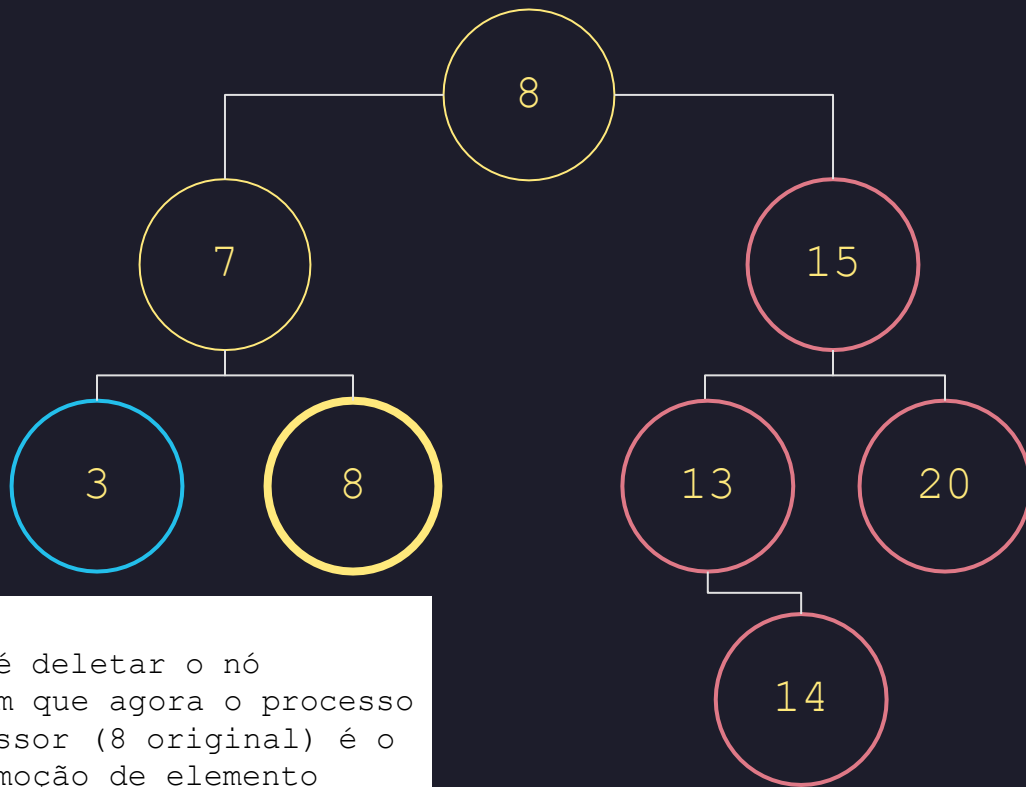


Removendo o 10:

O nó 8 é o valor menor mais próximo ao 10 que temos nesse exemplo. Então vamos trocar o **valor** do nó raiz (10) pelo **valor** do nó sucessor menor mais próximo (8). Percebam que o 8 fica duplicado na árvore, mas é isso mesmo.

MENOR

Caso 3: Remoção de um elemento com 2 filhos

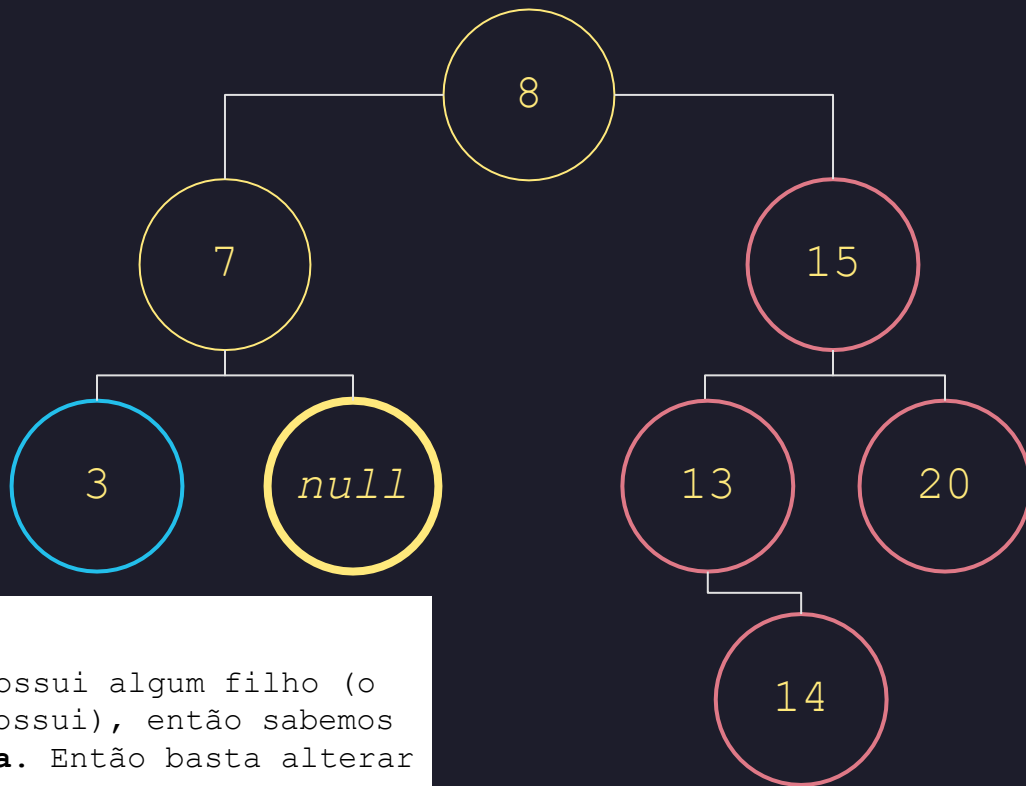


Removendo o 10:

O que fazemos agora é deletar o nó sucessor (8). E vejam que agora o processo de deletar o nó sucessor (8 original) é o mesmo do "Caso 1: remoção de elemento folha".

MENOR

Caso 3: Remoção de um elemento com 2 filhos

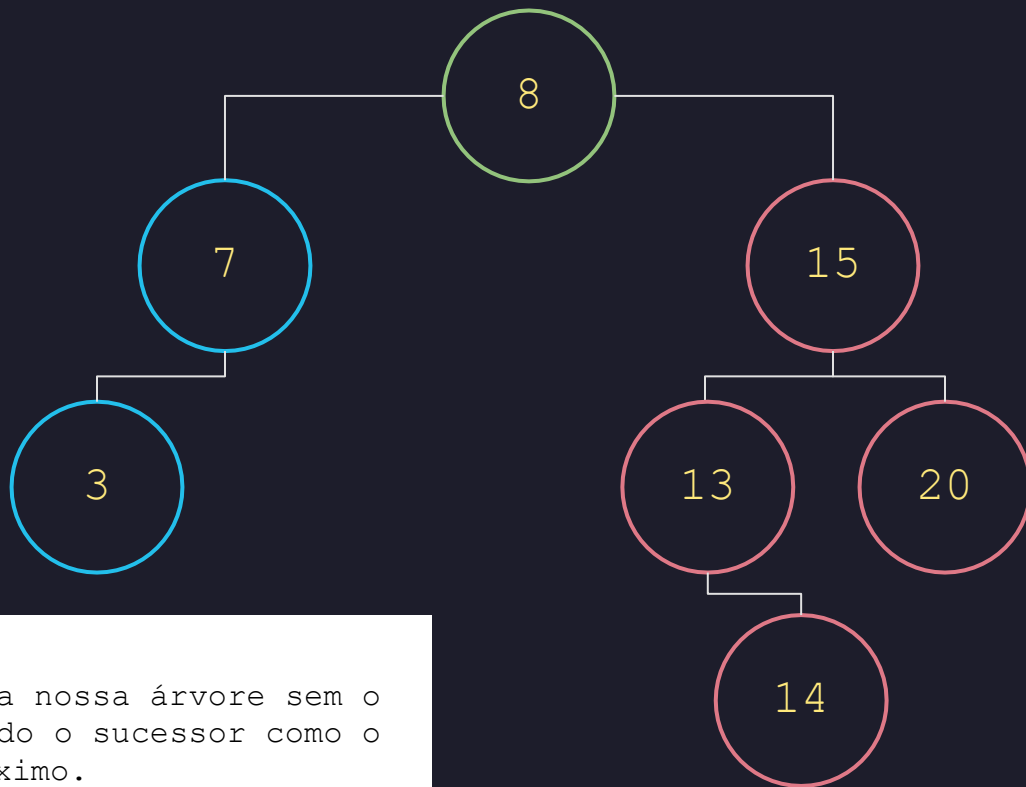


Removendo o 10:

Verificamos se ele possui algum filho (o que nesse caso não possui), então sabemos que o 8 é um nó **folha**. Então basta alterar o nó para *null*.

MENOR

Caso 3: Remoção de um elemento com 2 filhos



Removendo o 10:

Esse é o resultado da nossa árvore sem o valor 10, considerando o sucessor como o **menor valor** mais próximo.

MENOR